

Human Development Index (HDI) Predictor

Project Documentation

Project Description

The Human Development Index (HDI) Predictor is an interactive machine learning web platform designed to analyze, predict, and evaluate country-level human development tiers. The application utilizes a Random Forest Regressor model trained on a structured dataset of real country indicators, evaluating four critical dimensions of human capability:

- Health — Life expectancy at birth (years)
- Knowledge — Mean years of schooling
- Economic standard — Gross National Income per capita (PPP, US\$)
- Digital infrastructure — Internet users as a percentage of population

By shifting the evaluation focus from purely economic growth to human capabilities, this tool gives policymakers, researchers, and international organizations a broader perspective on global development. Users dynamically input country metrics to predict an exact composite HDI score and classify a nation into one of four tiers: Very High, High, Medium, or Low.

Scenarios

Scenario 1: Predicting Very High Human Development

A user inputs a life expectancy of 82, mean schooling of 13, GNI of 55,000, and internet usage of 95%. The RandomForestRegressor returns a score above 0.800, which `classify_hdi()` maps to Very High — confirming strong performance across all four development dimensions.

Scenario 2: Identifying Development Gaps in Emerging Economies

A policymaker enters life expectancy 68, mean schooling 7, GNI 8,000, internet 45%. The model returns a Medium HDI score (~0.59), indicating that targeted improvements in education or income could elevate the country to the High tier.

Scenario 3: Assessing Countries Requiring Development Intervention

A researcher evaluates life expectancy 56, mean schooling 3, GNI 1,300, internet 15%. The model predicts a Low HDI score (below 0.550), quantifying the acute development bottlenecks that justify prioritized humanitarian investment.

Technical Architecture

Frontend (User Interface Layer)

Dark-themed HTML5 interface with radial gradient CSS background. Jinja2 template engine dynamically renders the country dropdown and prediction result without page reload.

Backend Framework (Application Layer)

Python Flask server (app.py) handles dataset ingestion, form parsing via request.form.get(), safe type conversion, routing, and error handling.

Machine Learning Engine (Predictive Layer)

scikit-learn RandomForestRegressor with 300 estimators trained in-memory at startup on 53 countries from hdi_dataset.csv.

Pre-requisites

- Python 3.8+ — conditional logic and exception handling
- pandas, numpy, scikit-learn — data ingestion, DataFrame ops, and ML
- Flask — routing, request parsing, Jinja2 templating
- HTML/CSS — form validation, flexbox layout, gradient styling
- Command Line — pip install, venv activation, running Python scripts

Project Workflow

Milestone 1: Architecture Setup & Data Preparation

This milestone establishes the development environment, project folder layout, and the CSV dataset that drives the ML model.

Activity 1.1: Setup Python Virtual Environment

A local venv was created to isolate all project dependencies from the system Python installation, preventing version conflicts across projects.

```
python -m venv venv
venv\Scripts\activate      # Windows
pip install flask scikit-learn numpy pandas gunicorn
```

Activity 1.2: Configure Project Directory Structure

Directories are organized to cleanly separate backend logic, HTML templates, and dataset files. Flask requires HTML files to live in a folder named exactly templates/.

```
hdi_app/
app.py# Flask backend and ML engine
requirements.txt# dependency list for deployment
templates/
index.html# Jinja2 HTML frontend
data/
hdi_dataset.csv# training dataset (53 countries)
generate_dataset.py# script used to build the CSV
```

Activity 1.3: Gather & Save HDI Dataset

A structured dataset of 53 countries was assembled covering all four HDI tiers. Each row contains seven columns — country, life_expectancy, mean_schooling, gni_per_capita, internet_users, hdi_score, and hdi_tier — saved at data/hdi_dataset.csv.

Column	Description	Example
country	Country name (populates dropdown)	India
life_expectancy	Life expectancy at birth (years)	67.7
mean_schooling	Mean years of schooling completed	6.7
gni_per_capita	GNI per capita PPP (US\$)	6951
internet_users	% of population using internet	46.0
hdi_score	Computed composite HDI score	0.612
hdi_tier	Tier derived from score	Medium

Milestone 2: Core Machine Learning Engine Development

Covers building the predictive engine — data loading, feature/target isolation, and Random Forest training.

Activity 2.1: Data Load Pipeline with FileNotFoundError Guard

load_dataset() uses os.path.exists() to assert the CSV is present before calling pd.read_csv(). A missing file raises a descriptive FileNotFoundError immediately rather than crashing during training.

```
def load_dataset():
    if not os.path.exists(DATASET_PATH):
        raise FileNotFoundError(
            f'Dataset not found at {DATASET_PATH}.'
        )
    return pd.read_csv(DATASET_PATH)
```

Activity 2.2: Feature and Target Isolation

The DataFrame is split into X (four feature columns the user will input) and y (the hdi_score target column). This clean separation is required by the scikit-learn .fit(X, y) API.

```
#Independent Variables
feature_cols = ['life_expectancy', 'mean_schooling', 'gni_per_capita', 'internet_users']
```

```
X = df[feature_cols] # shape: (53, 4) <- inputs
X = pd.DataFrame(X)

#Dependent Variable
y = df[hdi_score] # shape: (53,) <- target
y = pd.DataFrame(y)
```

Activity 2.3: RandomForestRegressor Training

A RandomForestRegressor with n_estimators=300 and random_state=42 is instantiated and trained via .fit(X, y). The fixed random_state ensures identical results on every app restart. The trained model is stored globally for reuse.

```
def train_model(df):
    model = RandomForestRegressor(
        n_estimators=300, random_state=42
    )
    model.fit(X, y)
    return model
```

Milestone 3: app.py Development

Covers the full app.py logic — the classification function, Flask route, and prediction pipeline.

Activity 3.1: classify_hdi() — Threshold Logic Map

Converts the continuous float score from the model into one of four tier labels using UNDP's official HDI threshold cutoffs.

```
def classify_hdi(hdi_score):
    if hdi_score >= 0.800: return 'Very High'
    elif hdi_score >= 0.700: return 'High'
    elif hdi_score >= 0.550: return 'Medium'
    else: return 'Low'
```

Activity 3.2: Primary Application Route

index() handles GET (renders empty form) and POST (runs prediction) at the root URL /. result and score are passed back to the Jinja2 template as context variables.

```
@app.route('/', methods=['GET', 'POST'])
def index():
    result = None
    score = None
    if request.method == 'POST':
        ... # prediction pipeline runs here
    return render_template('index.html', countries=COUNTRIES, result=result, score=score)
```

Activity 3.3: try-except Prediction Pipeline

Inside the POST block, a try-except structure safely extracts form values, casts them to floats, structures them into a DataFrame, runs inference, and classifies the result.

```
try:
    # Step 1 — Extract form input strings
    life_exp = float(request.form.get('life_expectancy'))
    mean_school = float(request.form.get('mean_schooling'))
    gni = float(request.form.get('gni'))
    internet_pct = float(request.form.get('internet_users'))

    # Step 2 — Build single-row DataFrame
    features = pd.DataFrame([[life_exp, mean_school, gni, internet_pct]])
    columns=['life_expectancy','mean_schooling','gni_per_capita','internet_users'])

    # Step 3 — Run model inference and classify
    predicted_score = model.predict(features)[0]
    result = f'{classify_hdi(predicted_score)} HDI'
    score = round(predicted_score, 2)
except (TypeError, ValueError):
    result = 'Please fill in all fields with valid numbers.'
```

Milestone 4: Frontend Development

Covers the HTML input form, dynamic country dropdown, and conditional result display in index.html.

Activity 4.1: Input Form with required and step Enforcement

Each numeric input uses type='number', step='0.1' or step='1', and required to enforce client-side validation before any data reaches Flask.

```
<selectname='country'required>
...dropdown options...
</select>

<inputtype='number'step='0.1'name='life_expectancy'
placeholder='Enter life expectancy rate (range 39-89)'required>

<inputtype='number'step='1'name='gni'
placeholder='Enter GNI (range 740-178000)'required>
```

Activity 4.2: Jinja2 Country Dropdown Loop

The country list is generated dynamically via a Jinja2 for loop over the countries variable passed from Flask. Any new country added to the CSV auto-appears in the dropdown on the next restart.

```
<selectname='country'required>
<optionvalue="disabledselected">Select the name of the Country</option>
{% for c in countries %}
<optionvalue="{{ c }}">{{ c }}</option>
{% endfor %}
</select>
```

Activity 4.3: Conditional Result Display Block

The {% if result %} block only renders after a successful POST. It displays the tier label and rounded score injected by Flask, styled with a dark card and gold text.

```
{% if result %}
<divclass='result-box'>
<divclass='result-tier'>
  {{ result }}{% if score is not none %} {{ score }}{% endif %}
</div>
</div>
{% endif %}
```

Milestone 5: Deployment

Covers installing dependencies, running Flask locally, validating predictions, and deploying publicly.

Activity 5.1: Install Package Dependencies

All required packages are installed via pip. gunicorn is included for production deployment on Render.

```
cd C:\hdi_app
pip install flask scikit-learn numpy pandas gunicorn
```

Activity 5.2: Initialize Flask Runtime Locally

Running app.py triggers dataset loading and model training before the server starts. Expected terminal output confirms the dataset loaded successfully.

```
python app.py

# Expected output:
Loaded 53 countries from dataset.
* Running on http://127.0.0.1:5000

# For production (Render):
gunicorn app:app
```

Activity 5.3: Validate Outputs Across Test Cases

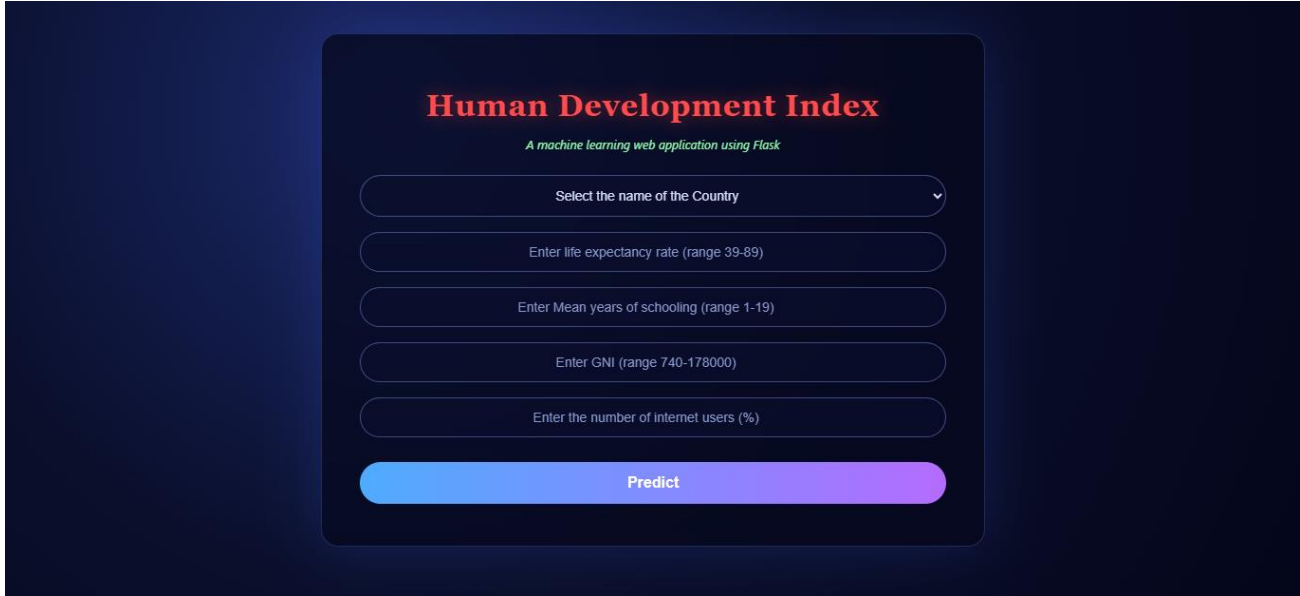
Submit the following parameter sets at <http://127.0.0.1:5000> to verify correct tier classifications across the full HDI range:

Life Exp	Mean School	GNI	Internet %	Expected Tier	Expected Score
82	13	55000	95	Very High HDI	~0.93–0.96
76	10	22000	80	High HDI	~0.72–0.78
68	7	8000	45	Medium HDI	~0.55–0.65
56	3	1300	15	Low HDI	~0.25–0.40

For public access, deploy to Render using the GitHub repository with build command `pip install -r requirements.txt` and start command `gunicorn app:app`.

Exploring the Website's Web Pages:

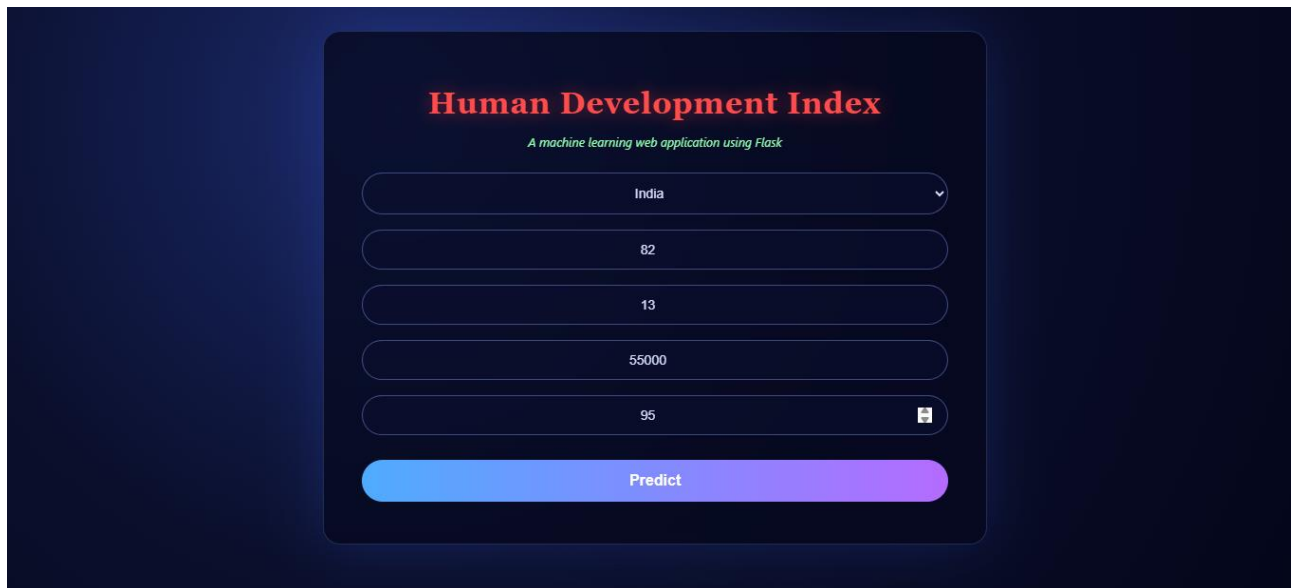
Home Page (Empty Form)



The screenshot shows a web application interface with a dark blue background. A central card contains the title "Human Development Index" in red, followed by the subtitle "A machine learning web application using Flask" in green. Below these are five input fields: a dropdown menu for "Select the name of the Country", and four text inputs for "Enter life expectancy rate (range 39-89)", "Enter Mean years of schooling (range 1-19)", "Enter GNI (range 740-178000)", and "Enter the number of Internet users (%)". At the bottom of the card is a large, rounded "Predict" button with a blue-to-purple gradient.

Home page of the Human Development Index Predictor web application. The interface displays a dark-themed card layout with the application title "Human Development Index" and subtitle "A machine learning web application using Flask." The form contains five input fields — a country dropdown, life expectancy, mean years of schooling, GNI, and internet users percentage — all empty and ready for user input. The gradient Predict button is visible at the bottom.

Form Filled with Input Values

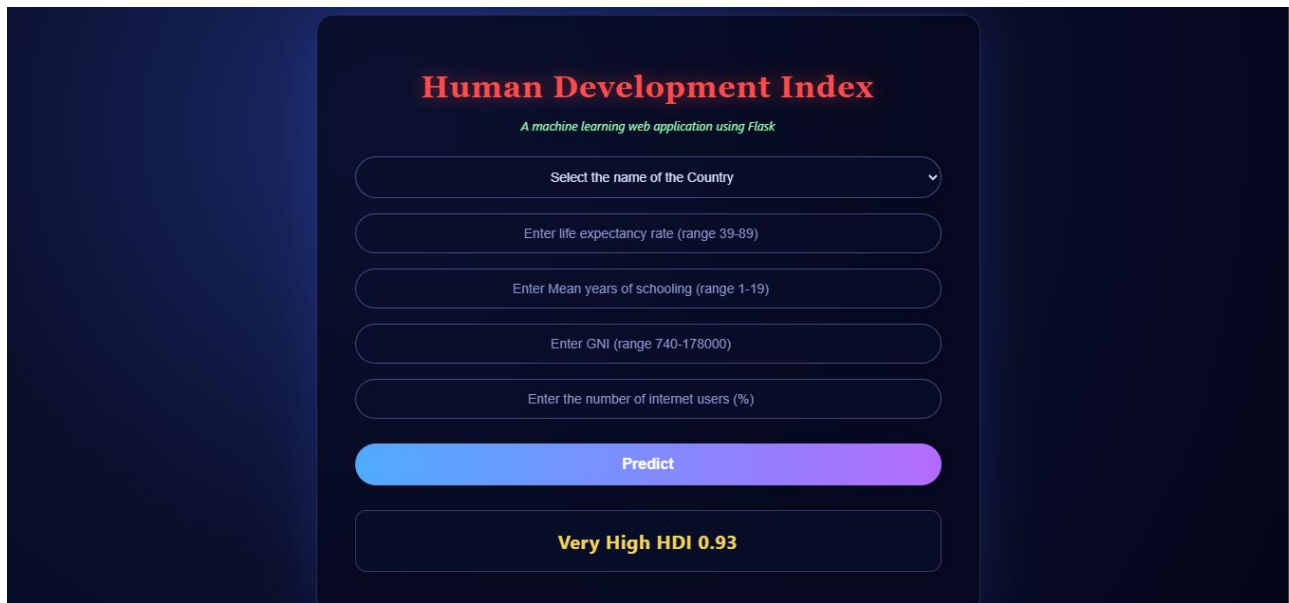


The screenshot shows a web application titled "Human Development Index" with the subtitle "A machine learning web application using Flask". The form contains five input fields with the following values: "India" (selected from a dropdown), "82", "13", "55000", and "95" (with a percentage icon). A "Predict" button is at the bottom.

Field	Value
Country	India
Life Expectancy	82
Mean Years of Schooling	13
GNI	55000
Internet Users	95%

The prediction form after the user has entered input values. The country selected is India, with a life expectancy of 82, mean years of schooling of 13, GNI of 55,000, and internet users of 95%. These values represent a high-performing country profile across all four development indicators. The user is ready to click the Predict button to submit the data to the Flask backend for model inference.

Prediction Result Displayed



The screenshot shows a web application titled "Human Development Index" in red text. Below the title is a subtitle "A machine learning web application using Flask" in green. The application features five input fields: a dropdown menu for "Select the name of the Country", and four text inputs for "Enter life expectancy rate (range 39-89)", "Enter Mean years of schooling (range 1-19)", "Enter GNI (range 740-178000)", and "Enter the number of internet users (%)". A prominent blue and purple "Predict" button is located below the inputs. At the bottom, a result box displays "Very High HDI 0.93" in gold-colored text.

The application after the user clicks Predict. The Flask backend extracts the four input values, structures them into a pandas DataFrame, passes them through the trained RandomForestRegressor model, and classifies the output using the `classify_hdi()` function. The result box at the bottom displays "Very High HDI 0.93" in gold-coloured text, correctly identifying the country as belonging to the Very High human development tier — confirming the model's prediction accuracy for high-indicator input profiles.

Conclusion

The Human Development Index (HDI) Predictor demonstrates how machine learning can be applied to multidimensional human development data to produce fast, interpretable predictions accessible through a simple web interface. By training a Random Forest ensemble on four standardized country-level indicators — life expectancy, mean years of schooling, GNI per capita, and internet penetration — the model captures the complex interplay between health, education, income, and digital access that defines a nation's development tier.

This multidimensional approach enables policymakers, researchers, healthcare professionals, and organizations to make informed decisions, design effective interventions, and allocate resources more efficiently. It supports strategies aimed at improving public health, reducing inequalities, enhancing social cohesion, and promoting sustainable development.

Ultimately, measuring well-being in a comprehensive manner contributes to building healthier, happier, and more resilient societies. By focusing on the factors that genuinely impact people's lives, it encourages balanced progress and helps create inclusive communities where individuals have the opportunity to achieve their full potential and enjoy a higher quality of life.

Looking ahead, the application has significant potential for expansion: integrating real-time UNDP data feeds, adding year-over-year trend visualizations, supporting multi-country comparison views, and incorporating additional indicators such as gender inequality or environmental sustainability metrics would further deepen its analytical value for global development stakeholders.